PITA: PREFERENCE-GUIDED INFERENCE-TIME ALIGNMENT FOR LLM POST-TRAINING

Anonymous authors
Paper under double-blind review

ABSTRACT

Inference-time alignment enables large language models (LLMs) to generate outputs aligned with end-user preferences without further training. Recent post-training methods achieve this by using small guidance models to modify token generation during inference. These methods typically optimize a reward function KL-regularized by the original LLM taken as the reference policy. A critical limitation, however, is their dependence on a pre-trained reward model, which requires fitting to human preference feedback, a potentially unstable process. In contrast, we introduce PITA, a novel framework that integrates preference feedback directly into the LLM's token generation, eliminating the need for a reward model. PITA learns a small preference-based guidance policy to modify token probabilities at inference time without LLM fine-tuning, bypassing the pre-trained reward model dependency. The problem is framed as identifying an underlying preference distribution, solved through iterative refinement of the preference-based guidance model. We evaluate PITA across diverse tasks, including mathematical reasoning, and sentiment classification, demonstrating its effectiveness in aligning LLM outputs with user preferences.

1 INTRODUCTION

Large language model (LLM) alignment traditionally relies on additional training, such as fine-tuning with human feedback, to ensure outputs conform to human desires. However, fine-tuning LLMs is computationally expensive and complex. This has motivated the study of inference-time alignment, where the pre-trained LLM's weights remain frozen, and alignment is achieved by steering the model's decoding process to meet end-user needs. Inference-time alignment aims to align generated outputs with user criteria (e.g., helpfulness, correctness, sentiment) without updating the LLM's parameters. By intervening only during generation, this approach avoids retraining overhead and enables the alignment of opaque models whose internal weights are inaccessible.

Recent post-training alignment methods employ small guidance models or value functions to influence token generation during decoding. These approaches frame decoding as an optimization problem, where the model maximizes a user-provided reward while maintaining proximity to its original style. They optimize a KL-regularized objective, balancing the base LLM's log-likelihood with the reward from the guidance model. The reward signal originates from a pre-trained model that scores candidate tokens based on their anticipated impact on the alignment of subsequent tokens with the reward criteria. These methods integrate the guidance model into the decoding loop, adjusting the next-token distribution at each step. Although computationally intensive, these techniques enhance alignment without modifying the LLM's weights.

Reward-model-guided alignment methods face several challenges. First, they assume the availability of a pre-trained reward model, which is often not available. Reward models are typically learned from human-labeled preference data. However, preference data might be noisy or inconsistent, hindering the accurate learning of a reward model that reflects true user intent. Moreover, training a reward model can be unstable and prone to brittleness, where minor inaccuracies in the model can cause the LLM to generate misaligned outputs. These issues highlight the difficulty of learning a reliable reward model from preference data, motivating the need to eliminate an explicit reward predictor.

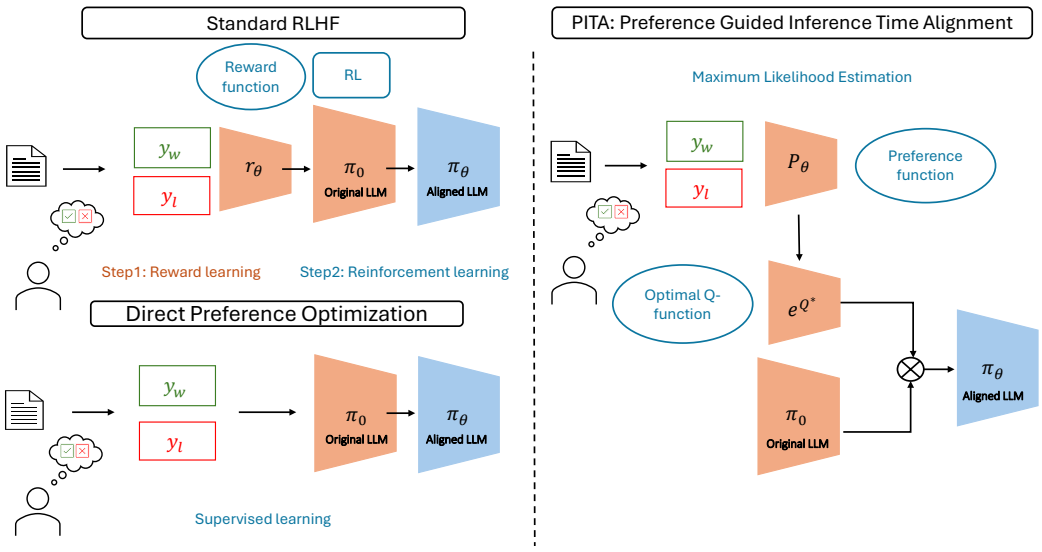

Figure 1: Overview of PITA training and inference time implementation. During training, our algorithm learns a Action-Value function Q^* directly from the preference information. At Inference, the output distribution of the original LLM, π_{ref}, is modified with exponentially weighted Q^*-values

In this work, we introduce **PITA: Preference-Guided Inference-Time Alignment**, a novel framework that eliminates the need for a reward model in LLM post-training. PITA directly uses preference feedback during inference to guide token generation. The key innovation is the use of a small, preference-based guidance policy that operates alongside a frozen LLM and adjusts token probabilities in real-time, without modifying the LLM's weights. This guidance policy is trained using user preference signals, significantly reducing the requirement for extensive human-labeled data and avoiding the complexity of reward model training.

PITA frames the alignment task as learning an underlying preference distribution over the model's outputs. It uses stochastic search and iterative refinement to identify tokens that align better with user preferences. During each decoding round, the guidance policy explores multiple continuations and updates based on user feedback, progressively refining the model's predictions. This process guides the model to converge towards outputs that better match user preferences, while keeping the base LLM unchanged. Figure 1 illustrates the differences between PITA and standard post-training.

We first provide a performance analysis of PITA, using the notion of a linear function approximator for the preference model. As the learned preference model aligns with the true one, PITA's guidance model aligns with the true reward-to-go, resulting in sub-linear regret. We then evaluate PITA on diverse tasks, including mathematical reasoning and sentiment alignment datasets. An example of the outputs generated by the reference LLM vs. PITA for a math reasoning problem from the GSM8K dataset is shown in Figure 2. Here, PITA aligns the model's steps with user-preferred solutions without fine-tuning, improving accuracy and consistency. In sentiment tasks, PITA ensures outputs meet user-defined sentiment or safety criteria, such as avoiding toxic language.

We compare PITA against two reward-based alignment schemes: Q# (Zhou et al., 2025) uses a known reward, and Q#-HF where we learn reward from preference data via maximum likelihood estimation, followed by alignment using Q#. PITA performs comparably to Q#, while outperforming the learned-reward-based alignment method (as well as the base LLM), demonstrating its superior efficiency and effectiveness in utilizing preference data. PITA achieves this level of performance without requiring a pre-trained reward model, highlighting its advantage in preference-guided inference-time alignment.

2 Related Work

Preference learning has a long history in RL and bandit literature (Akrour et al., 2012; Auer et al., 2014; Sadigh et al., 2017). Alongside the empirical success of training LLMs to generate outputs aligned with human preferences referred to as Reinforcement Learning with Human Feedback

Problem from GSM8K: Christina is planning a birthday party and needs .75 gift bags per invited guest, because 1/4 of attendees don't show up. She invited 16 friends. Gift bags are $2 each. How much will she spend?

Incorrect Solution from π^{ref}: Let's break this down step by step: 1. Christina invited 16 friends. 2. Since 1/4 of attendees don't show up, she'll only need to give gift bags to 3/4 of the invited guests. To find this amount, multiply 16 by 3/4: 16 × (3/4) = 12. 3. Since she needs 0.75 gift bags per invited guest, she'll need 0.75 × 12 = 9 gift bags. 4. Each gift bag costs $2, so the total cost will be 9 × $2 = $18. So, Christina will spend $\boxed{\$18}$ on gift bags.

Correct Solution from PITA: Since Christina invited 16 friends, and 1/4 of attendees don't show up, the number of friends who will actually attend the party is 16 - (1/4)*16 = 16 - 4 = 12. Since Christina needs 0.75 gift bags per invited guest, and she invited 16 friends, she needs 0.75*16 = 12 gift bags. The cost of each gift bag is $2, so the total cost of the gift bags is 12*2 = $24. Final Answer: The final answer is $\boxed{24}$. I hope it is correct.

Figure 2: Comparison of Incorrect Solution from the Reference model and the Correct Solution from PITA to a math reasoning problem.

(RLHF), the theoretical aspects of preference learning have been extensively studied (Chambers et al., 2021; Shah et al., 2016; Heckel et al., 2018). Reward-based methods have been shown to be unstable, and relative preferences are generally easier to obtain than absolute reward scores. Direct preference optimization (DPO) has emerged as a promising approach, addressing the inconsistencies of reward-based methods in reflecting true human preferences (Knox et al., 2022). Several works characterize the alignment problem as a general preference optimization objective, which is the focus of our work. For instance, Azar et al. (2024) define a general RLHF objective, with RLHF and DPO as specific examples. In Contrastive Preference Learning (CPL) (Hejna et al., 2023), a novel approach models human preferences based on regret rather than reward, enabling scalable, off-policy learning without requiring reward functions or reinforcement learning. Additionally, Ye et al. (2024) proposes a general RLHF framework without assuming a reward function or Bradley-Terry preferences, using a reverse-KL minimax game between LLMs, and introduces sample-efficient algorithms for both offline and online learning with empirical validation. An extended version of the related work is provided in Appendix A.

3 PRELIMINARIES

We begin by introducing the finite-horizon Markov Decision Process (MDP) framework that formalizes the generation process of language modeling.

A finite-horizon MDP is formulated as $\mathcal{M} = (\mathcal{P}, \mathcal{S}, \mathcal{A}, T, \rho)$, where $\mathcal{S}$ is a finite set of states, $\mathcal{A}$ is a set of actions, $\mathcal{P} : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ is the transition kernel, $T \in \mathbb{N}$ is the maximal length of the episode, and ρ denotes the starting state distribution. A policy in MDPs, denoted by $\pi : \mathcal{S} \to \Delta(\mathcal{A})$, maps each state to a distribution over actions. The dynamics of the MDP can be described as follows: Initially, the starting state s_0 is sampled from the initial distribution ρ. At each step t, the policy selects an action a_t based on the current state s_t. The process transitions to the next state s_{t+1}, which is sampled from the distribution $\mathcal{P}(\cdot \mid s_t, a_t)$. This process continues until a termination condition is met within the horizon length T.

LLMs as Deterministic MDPs: In text generation applications using an LLM, π is the LLM's policy to choose the next token (action) given the current sequence of tokens (state). The transition kernel is deterministic; each new state is obtained by concatenating the previous tokens with the current token chosen by the policy. Each sentence generated by the policy is termed a trajectory $\tau = (s_0, a_0, s_1, a_1, \cdots)$, where $s_0 := x$ is the initial instruction sampled from the starting state distribution ρ, and $a_t \sim \pi(\cdot|s_t)$. Therefore, at each step t, the state $s_t = (x|a_{1:t-1})$ consists of the instruction x and tokens generated up to $t-1$. The generation process ends when the LLM policy samples a special end-of-sequence token or the trajectory reaches a predefined length

limit, characterized by T. The score of a trajectory generated by the LLM is defined using a reward function $r(s,a) : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$. The reward function is typically sparse with $r(s_t, a_t) = 0$ for all $t \in [T-2]$, quantifying the desirability of the final output of the LLM for the given instruction x. An LLM policy can be learned using T-horizon reinforcement learning by maximizing the expected reward for a given policy defined as $J(\pi) = \mathbb{E}_{\tau\sim\pi}[R(\tau)] = \mathbb{E}_{x\sim\rho}[V^\pi(x)]$, where $V^\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^{T-1} r(s_t,a_t)|s_0 = s\right]$ is the value of a state s under policy π. Similarly, the state-action value function and the advantage function for a state-action pair (s,a) are given by $Q^\pi(s,a) = \mathbb{E}_\pi\left[\sum_{t=0}^{T-1} r(s_t,a_t)|s_0 = s, a_0 = a\right]$, and $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ respectively.

RLHF/DPO: RLHF and DPO are a few commonly used approaches to align LLMs with human preference data. The RLHF pipeline consists of three steps: a supervised fine-tuning phase where a pre-trained LLM is fine-tuned with supervised learning for downstream task(s) to obtain a model π_{ref}, a reward modeling phase to estimate a reward function for the LLMs generations, and a fine-tuning phase to find the optimal policy which maximizes the expected reward while not deviating too far from the reference model π_{ref}.

Reward Modeling: The reference model π_{ref} is prompted with an instruction x to obtain an answer pair (y_1, y_2). The completions are presented to human annotators who express their preference for one answer over the other, denoted as $y_w \succ y_l|x$ where y_w and y_l denote the preferredanswer. The preferences are assumed to be generated according to the Bradley-Terry model (Bradley and Terry, 1952), given by

$$\mathcal{P}^*(y_w \succ y_l) = \sigma(r^*(x, y_w) - r^*(x, y_l)). \tag{1}$$

where $r^*(\cdot,\cdot)$ is a latent reward model unknown to the LLM. Assuming access to an offline dataset $\mathcal{D} = \{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}_{i=1}^n$ of preference information generated from $\mathcal{P}^*$, the reward model is parameterized as $r_\theta(x,y)$ and estimated via maximum likelihood. The estimation problem is formalized as a binary classification problem with the negative log-likelihood loss given by

$$\mathcal{L}(r_\theta, \mathcal{D}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma(r_\theta(x,y_w) - r_\theta(x,y_l))\right] \tag{2}$$

RL Fine-Tuning: The reference model π_{ref} is fine-tuned with the learned reward function r_θ, which is posed as the following KL-regularized constraint optimization problem:

$$\max_\pi \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi(y|x)}\left[r_\theta(x,y)\right] - \eta\mathrm{KL}\left[\pi(y \mid x) \,\|\, \pi_{\text{ref}}(y \mid x)\right] \tag{3}$$

where η is a parameter that controls the deviation of the learned π from the reference policy π_{ref}, and the KL divergence is defined as $\mathrm{KL}(p||q) = \mathbb{E}_{z\sim p}\left[\log p(z)/q(z)\right]$.

DPO: In Rafailov et al. (2023), the RL fine-tuning step is solved as a supervised learning problem to directly learn the optimal policy using preference information. The key idea is that the optimal policy, as obtained by solving the KL-constrained reward maximization objective, is a function of the implicit reward model and has the following form:

$$\pi_r^*(y \mid x) = \frac{1}{Z(x)}\pi_{\text{ref}}(y \mid x)\exp\left(\frac{1}{\eta}r^*(x,y)\right) \tag{4}$$

where $Z(x) = \sum_y \pi_{\text{ref}}(y|x)\exp(\frac{1}{\eta}r^*(x,y))$ is the partition function. Although estimation of the partition function is difficult, DPO circumvents this challenge by expressing the BT preference model as a function of the optimal policy π^* given by

$$\mathcal{P}^*(y_1 \succ y_2 \mid x) = \frac{1}{1 + \exp\left(\eta\log\frac{\pi^*(y_2|x)}{\pi_{\text{ref}}(y_2|x)} - \eta\log\frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)}\right)} \tag{5}$$

Similar to the reward modeling approach, under the above reformulation of the preference model, the policy objective is defined as:

$$\mathcal{L}_{\text{DPO}}(\pi;\pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\eta\log\frac{\pi(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \eta\log\frac{\pi(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]. \tag{6}$$

4 PITA: PREFERENCE-GUIDED INFERENCE-TIME ALIGNMENT

In this section, we present our algorithm for test-time scaling of LLMs using a preference model. The key idea is to leverage the structure of the optimal policy under π^* as a function of the state-action value function Q^* in deterministic MDPs (Zhou et al., 2025). This framework applies specifically to LLM fine-tuning. To facilitate understanding, we present the relevant results.

For a given parameter $\eta > 0$, the soft-value function $V^{\pi,\eta}$ of a policy π is defined as:

$$V^{\pi,\eta}(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{T-1} r(s_t, a_t) - \eta \, \mathrm{KL}(\pi(s_t) \parallel \pi_{\mathrm{ref}}(s_t)) \Big| s_0 = s \right] \tag{7}$$

It can be shown that the KL-regularized RL objective is solvable using the soft Bellman equations. In particular, the optimal policy is given by Rafailov et al. (2024):

$$\begin{aligned} V_T^{\star,\eta}(s) &= 0, \\ Q_t^{\star,\eta}(s,a) &= r(s,a) + \mathbb{E}_{s' \sim P(\cdot|s,a)} \left[V_{t+1}^{\star,\eta}(s') \right], \\ \pi^{\star,\eta}(a_t \mid s_t) &\propto \pi_{\mathrm{ref}}(a_t \mid s_t) \exp\left(\eta^{-1} Q_t^{\star,\eta}(s_t, a_t) \right), \\ V_t^{\star,\eta}(s) &= \eta \ln \mathbb{E}_{a \sim \pi_{\mathrm{ref}}(\cdot|s)} \left[\exp\left(\eta^{-1} Q_t^{\star,\eta}(s,a) \right) \right]. \end{aligned} \tag{8}$$

Here, $Q^{\star,\eta}$ represents the soft-state-action value function under the optimal policy π^*. We specialize our discussion to the deterministic transition structure of the LLM generative process. In the sparse reward setting, $V^{\pi,\eta}, Q^{\pi,\eta}$ are functions of the conditional reward distributions of the generations under π_{ref} (Zhou et al., 2025). Specifically:

$$V_t^{\star,\eta}(s) = \eta \ln \mathbb{E}_{\pi_{\mathrm{ref}}} \left[\exp\left(\eta^{-1} r(s_{T-1}, a_{T-1}) \right) \big| s_t = s \right], \tag{9}$$

$$Q_t^{\star,\eta}(s,a) = \eta \ln \mathbb{E}_{\pi_{\mathrm{ref}}} \left[\exp\left(\eta^{-1} r(s_{T-1}, a_{T-1}) \right) \big| s_t = s, a_t = a \right]. \tag{10}$$

For the sake of completeness, we present the proof of the above results in Appendix B. The Q# approach utilizes a policy of the form given in (8) with $Q_t^{\star,\eta}(s,a)$ as given in (10).

Preference-based Q Value function: Unlike Q#, the goal of PITA is to directly learn an optimal LLM policy using preference data.

We first derive the soft-state-action value function and the value function in terms of the preference distribution. For a given context x, let y_x^{ref} be a unique *reference* completion starting from state x. The preference distribution of any completion y_x given x, sampled according to a policy π over y_x^{ref}, is given by $\mathcal{P}^*(y_x \succ y_x^{\mathrm{ref}})$.

Theorem 1. *Let $\Psi(x) \triangleq \log \frac{x}{(1-x)}$. Then, assuming the BT preference model, we have*

$$\begin{aligned} V_t^{\star,\eta}(s) &= r(y_s^{\mathrm{ref}}) + \eta \ln \mathbb{E}_{y_s \sim \pi_{ref}} \left[\exp\left(\eta^{-1} \Psi(\mathcal{P}^*(y_s \succ y_s^{\mathrm{ref}})) \right) \big| s_t = s \right], \\ Q_t^{\star,\eta}(s,a) &= r(y_s^{\mathrm{ref}}) + \eta \ln \mathbb{E}_{y_{s'} \sim \pi_{ref}} \left[\exp\left(\eta^{-1} \Psi(\mathcal{P}^*(y_{s'} \succ y_s^{\mathrm{ref}})) \right) \big| s_t = s, a_t = a \right], \end{aligned} \tag{11}$$

where $s' = (s|a)$ is the state obtained by concatenating a to state s.

The proof of Theorem 1 is provided in Appendix B. The above theorem shows that the expressions $Q_t^{\star,\eta}, V_t^{\star,\eta}$ are functionals of the preference distribution of completions according to the reference policy π_{ref} with an offset term. While the reward function r^* is unknown, the optimal policy is a function of the advantage function, making it sufficient to evaluate the preference distribution. The reference completion for any intermediate state s must be unique. In the rest of the discussion, we choose y_s^{ref} to be the completion obtained via greedy decoding using policy π_{ref}.

Preference distribution over greedy decoding: The preference probability $\mathcal{P}^*(y_x \succ y_x^{\mathrm{ref}})$ represents the expected number of 'wins' of a completion over greedy decoding for a given context. Since the reference greedy completion y_x^{ref} is unique, using the reward-equivalence property of the Bradley-Terry model, we derive the following lemma.

Lemma 1. *Let r^* be the implicit reward function. Then, for any given context x and policy π, there exists a reward function $\tilde{r}$ such that the preference probability of generations $y_x \sim \pi$ over the greedy completion is given by $\sigma(\tilde{r}(y))$.*

The proof of Lemma 1 is given in Appendix B. The above reward equivalence is crucial for developing our algorithm. It states that the preference probability is solely a function of the completion and context. We can parameterize $\mathcal{P}^{\theta}(y_x)$ using a neural network, which estimates the expected wins $\mathcal{P}^*(y_x \succ y_x^{\text{ref}})$ over greedy decoding. From the optimal policy derived from the soft-Bellman equations, we have:

$$\pi^{\star,\eta}(a \mid s) \propto \pi_{\text{ref}}(a \mid s)\mathbb{E}_{y_{s'}\sim\pi_{\text{ref}}}\left[\exp\left(\eta^{-1}\Psi(\mathcal{P}^*(y_{s'} \succ y_s^{\text{ref}}))\right)\middle| s_t = s, a_t = a\right] \tag{12}$$

This naturally motivates us to consider parametrized policies of the form π^{θ}, such as:

$$\pi^{\theta,\eta}(a \mid s) \propto \pi_{\text{ref}}(a \mid s)\mathbb{E}_{y_{s'}\sim\pi_{\text{ref}}}\left[\exp\left(\eta^{-1}\Psi(\mathcal{P}^{\theta}(y_{s'} \succ y_s^{\text{ref}}))\right)\middle| s_t = s, a_t = a\right] \tag{13}$$

where $s' = (s|a)$ is the state obtained by concatenating s with action a.

4.1 Algorithm

In this section, we present PITA, a preference-guided inference-time alignment algorithm for solving the KL-regularized RLHF objective. PITA is an iterative algorithm where progressively better policies are learned using a parametrized preference function. We describe the PITA algorithm in the context of LLMs, and in Section 5, we provide convergence guarantees under linear reward approximation.

Note that the optimal policy is obtained by guiding generations from the reference policy π_{ref} using the regularized soft-action value function Q^*. However, evaluating Q^* is challenging unless the distribution over preference functions for each generation under π_{ref} is known *a priori*. Assuming a parametric family, one can apply distributional techniques, such as maximum likelihood estimation, for parameter inference. To address this, we adopt certainty equivalence as an approximation in our practical implementation.

Let $\hat{\mathcal{P}}^{\theta} : \mathcal{X} \to [0, 1]$ be the preference function that maps a given state $x := (s, a) \in \mathcal{X}$ to the expected preference under π_{ref}, i.e. $\mathbb{E}_{y_x\sim\pi_{\text{ref}}}[\mathcal{P}^*(y_x \succ y_s^{\text{ref}})]$. The natural loss function is the binary cross-entropy loss (BCE):

$$\mathcal{L}(o_y, \hat{\mathcal{P}}) = -o_y \log \hat{\mathcal{P}} - (1 - o_y) \log (1 - \hat{\mathcal{P}}) \tag{14}$$

where $o_y = \mathbf{1}_{y\succ y^{\text{ref}}}$ is the indicator function of the completion y preferred over the reference completion y^{ref}.

The algorithm iteratively updates the estimator parameters θ using new preference information collected by the induced policy $\pi_k \triangleq \pi^{\theta_k,\eta}$. Specifically, the data collection strategy follows a rollout step using π^k for t steps until a context s_t is sampled. Then, a completion is randomly sampled starting from this context along with greedy decoding using π_{ref}. The preference observed between the two completions is a sample from $\mathcal{P}^*$. These preference samples are added to the dataset, and parameters are updated via gradient descent. The procedure is repeated until convergence. The pseudocode for the algorithm is detailed in Algorithm 1.

Algorithm 1 Preference-Guided Inference-Time Alignment (PITA)

Input: Reference policy π^{ref}. Initialize θ_0 and dataset $\mathcal{D} = \emptyset$.
for $k = 1, 2, \ldots$ until convergence **do**
  Let $\pi^{k-1} \leftarrow \pi^{\theta_{k-1},\eta}$ be the policy induced by $\hat{\mathcal{P}}^{\theta_{k-1}}$.
  for $i = 1, 2, \ldots, N$ **do**
    Sample a rollout step uniformly: $t \sim \text{Unif}([T])$.
    Roll in with π^{k-1} for t steps to obtain the context s_t.
    Sample the reference response $y_{s_t}^{\text{ref}}$ starting from context s_t using greedy decoding.
    Sample M responses $y_{s_t}^1, \ldots, y_{s_t}^M$ with π^{ref} starting from s_t.
    Add $(s_t, y_{s_t}^i, y_{s_t}^{\text{ref}}, \mathbf{1}_{y_i\succ y_{s_t}^{\text{ref}}})$ to $\mathcal{D}$ for all $i \in [M]$.
  end for
  Update θ^k using Maximum Likelihood Estimation on the aggregated data:

$$\theta^k \leftarrow \arg\min_{\theta} \mathbb{E}_{(x,y,y')\sim\mathcal{D}}\left[\mathcal{L}(\mathbf{1}_{y\succ y'}, \hat{\mathcal{P}}^{\theta})\right]$$

end for
Output: Final θ^k. =0

We note that our algorithm belongs to the class of value-based algorithms for post-training LLMs (Mudgal et al., 2023; Han et al., 2024; Zhou et al., 2025). In methods like CD (Mudgal et al., 2023) and VAS (Han et al., 2024), the learned Q-function is a non-regularized version of the optimal Q^*. In contrast, Q# (Zhou et al., 2025) aims to learn the optimal state-action value function using distributional reinforcement learning (Bellemare et al., 2023). Our iterative algorithm shares design principles with Q#, but it learns the optimal action-value function directly from preference data, eliminating the need for a pre-trained reward model.

5 Theoretical Results on PITA Performance

In this section, we present theoretical performance guarantees for PITA. We focus on the case of a linear reward class. All proofs appear in Appendix C.

Assumption 1. *The reward lies in the family of linear functions $r_\theta(y) = \theta^\top \Phi(y)$ for some known Φ with $\sup \|\Phi(\cdot)\|_2 \leq L$. Let $\theta^\star$ be the true parameter. To ensure the identifiability of $\theta^\star$, we assume $\theta^\star \in \Theta_B$, where*

$$\Theta_B = \left\{\theta \in \mathbb{R}^d \mid \langle \mathbf{1}, \theta \rangle = 0, \|\theta\|_2 \leq B\right\}.$$

5.1 Preference Modeling in PITA

We denote by $\Phi(x)$ the d-dimensional embedding of a state x, where $\Phi : \mathcal{X} \to \mathbb{R}^d$. The embedding function is assumed to be known to the learner. In the context of LLMs, Φ is obtained from the hidden representation of the second-to-last layer of the model. The set of all completions starting from a given state x is denoted by $\mathcal{Y}_x^\pi = \{y : y \sim \pi(\cdot|x)\}$. Thus, the preference distribution $\mathcal{P}^*(y_x)$ for a completion $y_x \in \mathcal{Y}_x^\pi$ is given by

$$\mathcal{P}^*(y_x) = \mathcal{P}^*(y_x \succ y_x^\pi) = \sigma(\langle \theta^*, \Phi(y_x)\rangle - \langle \theta^*, \Phi(y_x^\pi)\rangle), \tag{15}$$

where y_x^π is the greedy completion under π. Since the greedy completion is unique for a fixed policy π and starting context x, we can rewrite the parametric class of preference distributions for PITA as:

$$\mathcal{P}^\theta(y_x) = \sigma(\langle \theta, \Phi(y_x) - \Phi(y_x^{\pi_{\text{ref}}})\rangle) \tag{16}$$

Since $y_x^{\pi_{\text{ref}}}$ is fixed, we can, without loss of generality, renormalize $\Phi(y_x^{\pi_{\text{ref}}})$ to 0, as all preference data for a given x is measured relative to $y_x^{\pi_{\text{ref}}}$. Thus, we have:

$$\mathcal{P}^\theta(y_x) = \sigma(\langle \theta, \Phi(y_x)\rangle). \tag{17}$$

Here, $\theta \in \Theta_B$, and $y_x \in \mathcal{Y}_x^{\pi_{\text{ref}}}$. In the following, we omit the context x from our notation, as the results apply on a per-context basis.

Maximum Likelihood Estimation (MLE). In the above preference modeling, a natural way to compute an estimator θ_k given completion pairs $\{(x_i, y_i, y_i^{\text{ref}})\}_{i=1}^n$ and their preference feedback values $\{o_i\}_{i=1}^n$ is via maximum likelihood estimation (MLE). MLE minimizes the negative log-likelihood function, defined as:

$$\hat{\theta} \in \arg\min_{\theta \in \Theta_B} \mathcal{L}_\mathcal{D}(\theta), \tag{18}$$

$$\mathcal{L}_\mathcal{D}(\theta) = -\sum_{i=1}^n o_i \log \sigma(\langle \theta, \Phi(y_i)\rangle) + (1 - o_i) \cdot \log 1 - \sigma(\langle \theta, \Phi(y_i)\rangle). \tag{19}$$

Following results from dueling bandits and reinforcement learning (Faury et al., 2020; Shah et al., 2016; Pacchiano et al., 2021), we have the following result on convergence of the MLE.

Lemma 2. *Under Assumption 1, for any $\lambda > 0$, with probability at least $1 - \delta$, we have*

$$\left\|\hat{\theta} - \theta^*\right\|_{\Sigma_\mathcal{D} + \lambda I} \leq C \cdot \sqrt{\frac{d + \log(1/\delta)}{\gamma^2 n} + \lambda B^2}. \tag{20}$$

where $\Sigma_\mathcal{D} = \frac{1}{n}\sum_{i=1}^n \Phi(y_i)\Phi(y_i)^\top$, $\gamma = \frac{1}{2+\exp(-LB)+\exp(LB)}$.

5.2 Regret Bound

We now state our main PAC result. We make the following assumption on the boundedness of the embedding function.

Assumption 2. *Consider the space of all policies of the form* $\Pi_\theta \triangleq \{\pi_\theta : \pi_\theta \propto \pi_{\text{ref}} \exp(\eta^{-1}\Phi)\}$. *We assume* $\left\|(\Sigma_{\mathcal{D}} + \lambda I)^{-1/2}\mathbb{E}_{y\sim\pi}[\Phi(y)]\right\|_2$ *is bounded for all* $\pi \in \Pi_\theta$.

Then we have the following regret bound on the performance of PITA.

Theorem 2. *For any* $\lambda > 0$, *with probability at least* $1 - \delta$,

$$\sum_{k=1}^{K}(V^{\star,\eta} - V^{\theta_k,\eta}) \leq C\sum_{k=1}^{K}\cdot\left(\sqrt{\frac{d+\log(1/\delta)}{\gamma^2 n_k} + \lambda B^2}\cdot\sup\left\|(\Sigma_{\mathcal{D}} + \lambda I)^{-1/2}\mathbb{E}_{y\sim\pi}[\Phi(y)]\right\|_2\right), \tag{21}$$

where n_k *is the number of preference samples collected up to iteration* k, *and* $\pi \in \Pi_\theta$.

Note that Theorem 2 demonstrates sub-linear regret with respect to the number of samples used for maximum likelihood estimation. In Zhu et al. (2023), the authors show the sub-optimality of policies estimated using a pessimistic MLE algorithm under the same linear reward model as ours. While standard concentration bounds for MLE estimators in linear reward settings are well-established, our convergence analysis follows a substantially different approach. The key insight is that the optimal soft-action value function Q^* can be expressed as a simple functional of preference data generated by a fixed reference policy. This transformation makes the analysis significantly more straightforward.

6 Experimental Evaluation

We empirically evaluate the proposed **PITA** algorithm for mathematical reasoning tasks, sentiment generation, and compare it against a strong reward-model baseline. Our study focuses on three key questions: **(Q1)** Does PITA improve task accuracy (*pass@1* and *maj@8*) over a baseline using post-training Q# with a learned preference reward model? **(Q2)** Does the KL divergence to the reference policy remain controlled? **(Q3)** How well does the learned reward model discriminate between correct and incorrect generations?

In this section, we provide an overview of the experimental results, while additional details on datasets, training, and evaluation criteria are provided in Appendix. We evaluate our algorithm for the star graph reasoning, and IMDB sentiment generation tasks in appendix D. We provide ablation study and additional training details for math reasoning in appendix section E. All experiments were run on machines equipped with a single NVIDIA A100 80 GB GPU.

6.1 Experimental Setup

Models and Datasets. We perform PITA training using the following models: Meta-Llama-3-8B-Instruct, Meta-Llama-3.2-1B-Instruct, TinyLlama-1.1B-Chat-v1.0, and GPT-2 small and medium models. We evaluate our algorithm on three diverse tasks with varying difficulty levels: arithmetic reasoning (GSM8K Cobbe et al. (2021)), star-graph reasoning Bachmann and Nagarajan (2024), and sentiment generation Chaudhary et al. (2025). We use the Llama 3 Grattafiori et al. (2024) family of models to train our preference function, as they are highly competitive in arithmetic reasoning tasks Zhou et al. (2025). The GPT-2 small and medium models are used for training on the star-graph reasoning task, following the framework in Bachmann and Nagarajan (2024). For sentiment generation, we use the TinyLlama 1.1B model, following Chaudhary et al. (2025).

Metrics. We report *pass@1* (exact-match accuracy of the highest-ranking sample), *maj@8* (majority-vote accuracy over 8 samples), and the forward KL divergence $\text{KL}(\pi \,\|\, \pi_{\text{ref}})$ computed on the GSM8K test set. Lower KL values indicate closer adherence to the reference policy.

6.2 Algorithms

We compare the performance of the following algorithms against the base reference model π_{ref}:

- **Q#**: The algorithm described in Zhou et al. (2025), trained directly using access to true binary reward labels.
- **Q#-HF**: A *two-stage* procedure in which a reward model is first trained from preference pairs (so-called "human feedback"), and this learned reward is then used in the Q# algorithm to obtain the final policy.
- **PITA (ours)**: The algorithm described in Section 4, which learns a soft Q-function directly from preference data and induces the final policy without requiring a separate reward model.

6.3 Performance on GSM8K

We present our results for the performance of our algorithm on the arithmetic reasoning dataset GSM8K Cobbe et al. (2021). GSM8K (Grade School Math 8K) is a dataset containing 8.5K high-quality, linguistically diverse math word problems aimed at the grade-school level. These problems, designed for question-answering tasks, require 2 to 8 steps and involve basic arithmetic operations. For test set evaluation, we use the entire GSM8K test set to evaluate the performance of the trained PITA model Lightman et al. (2023); Wang et al. (2023).

Reward-Model Calibration in Q#-HF: We first train the reward model needed by Q#-HF as an intermediate step before using the Q# algorithm to obtain a policy. We utilize the full preference data set in order to train this reward model. Figure 3 contrasts reward scores assigned by the BT model to correct versus incorrect generations produced by the baseline. A larger separation indicates stronger alignment between the reward and task success. The performance of Q#-HF is highly sensitive to the reward training process, as we see in an ablation study of reward training from preference data presented in Appendix E.

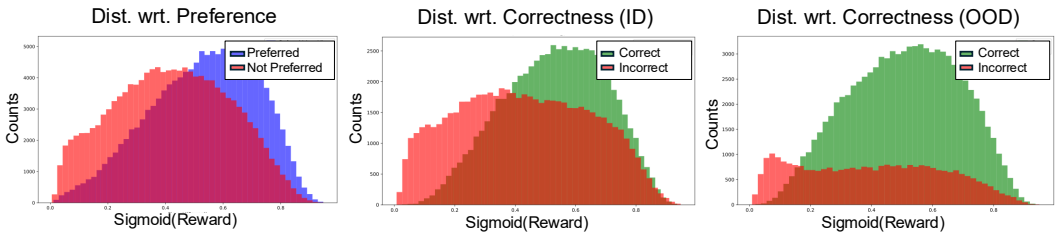

Figure 3: Distribution of learned reward scores w.r.t. preferred/correct (green) and not preferred/incorrect (red) for ID (in-dist.)/OOD (out-of-dist.) data. The reward function training is brittle and sensitive to the amount of data used.

Main Results: In Table 1, we compare the performance of PITA and other value-guided algorithms to the reference model π_{ref}, which is chosen to be the Llama 3 8B model. We observe that PITA significantly outperforms Llama 3 on both **pass@1** accuracy and **maj@8** evaluation metrics. Additionally, the performance gap between PITA and Q# appears to be small. Training details for PITA are provided in Appendix D. Thus, Table 1 shows that PITA attains performance on par with Q# even without access to true reward labels.

Despite using a learned reward model with full availability of preference data, Q#-HF fails to perform as well as PITA, highlighting the brittleness of reward estimation in this setting.

Additionally, the KL divergence of PITA is higher than that of Q# and Q#-HF. This is because the language model used to provide preference feedback in PITA is OpenMathInstruct-2 Toshniwal et al. (2024), which is distinct from the rule-based reward function score used in Q#. This highlights the tradeoff between alignment with the reference policy and the flexibility of preference-based feedback.

Table 1: GSM8K accuracy (%) and KL divergence.

Algorithm	**pass@1**	**maj1@8**	**KL↓**
π_{ref}	69.1	85.8	-
Q#	78.4	88.1	2.65
Q#-HF	67.23	78.09	2.48
PITA (ours)	77.11	86.20	6.15

Discussion: We observe that PITA significantly outperforms the Q#-HF baseline on both accuracy metrics. The poor performance of Q#-HF can be attributed to inconsistencies in the reward estimation process, which failed to provide stable guidance during training. This demonstrates that iterative policy optimization with sparse rewards, as used in PITA, is more sample-efficient than single-stage MLE fine-tuning. PITA's approach not only improves task performance but also reduces the need for large amounts of labeled preference data, making it more computationally efficient. Furthermore, PITA's ability to maintain alignment with the reference policy while improving accuracy highlights its robustness in real-world applications. Additional insights are presented in Appendix D.

6.4 Training-Agnostic Dependence on the KL Regularization Coefficient

The KL-regularization coefficient η plays a central role in shaping the behavior of the optimal policy at inference time. Recall that the policy derived under PITA (Eq. 13) depends only on the reference model π_{ref} and the learned preference function $\mathcal{P}^{\theta}$.

A key property of PITA is that its training process is agnostic to the choice of η. The algorithm focuses solely on learning a preference function, which is independent of the KL regularization. As a result, η acts purely as a tunable inference-time parameter: by adjusting its value, one can interpolate between policies closer to the reference model or more strongly guided by the preference function. Importantly, this flexibility comes without sacrificing accuracy gains, since the preference function itself remains invariant to η.

This property makes PITA especially practical in deployment. In practice, one may directly plug in off-the-shelf preference models and adjust η at inference to achieve the desired trade-off between fidelity to the reference model and alignment with preference feedback. In table 2, we present empirical results highlighting how varying η at inference time produces models with different effective KL-divergences from the reference policy, while maintaining consistent alignment benefits.

Table 2: PITA Model accuracy for different η

η	**pass@1**	**maj1@8**	**KL**$\downarrow$
0.5	66.04	78.92	2.37
1.0	68.90	79.98	2.58
4.0	76.70	86.12	4.44
6.0	**77.11**	**86.20**	**6.15**
8.0	76.46	87.26	8.59
10.0	75.91	88.80	21.67

7 Conclusion

In this work, we presented **PITA: Preference-Guided Inference-Time Alignment**, a novel framework for aligning large language models (LLMs) with user preferences during inference, without the need for extensive fine-tuning or pre-trained reward models. By directly integrating preference feedback into the decoding process, PITA offers a data-efficient, computationally lighter alternative to traditional alignment methods. We demonstrated its effectiveness across diverse tasks, such as mathematical reasoning and sentiment classification, achieving performance comparable to reward-based alignment methods, while outperforming those that require learning a reward model from preference data. Limitations to our approach include the need for high-quality preference data, and computational demands of the iterative nature of the refinement process. Additionally, while PITA eliminates the need for pre-trained reward models, it still relies on a reference policy.

Code Submission

Our code is available at `https://anonymous.4open.science/r/pref-sharp-D223`

A Additional Related Work

We provide an expanded discussion of related work omitted from the main paper.

Alignment: Large language models (LLMs) have demonstrated emerging capabilities in advanced natural language tasks through the use of **attention mechanisms** and the **transformer architecture** (Vaswani et al., 2017; Radford et al., 2019; Brown et al., 2020; Devlin et al., 2018; Bubeck et al., 2023). In the context of LLMs, the goal is to fine-tune the responses generated for downstream tasks. **Reinforcement Learning from Human Feedback (RLHF)** (Christiano et al., 2017; Ziegler et al., 2019; Stiennon et al., 2020; Ouyang et al., 2022) has become a significant technique in aligning LLMs with human values and preferences. A key aspect of the RLHF approach is the use of a **ground-truth reward model** (either a pre-trained model from preference data or labels from supervised data).

An alternative approach for utilizing preference information is **Direct Preference Optimization (DPO)** (Rafailov et al., 2023), which updates the model's policy directly using preference data. Unlike RLHF, DPO avoids reliance on a separate reward model by learning from relative preferences between pairs of outputs. This approach enables more direct control over model behavior, without the complexities and instability often associated with reward-based methods. The approach has been extended to relate DPO to learning an optimal soft Q-function (Rafailov et al., 2024), as well as more general preference functions (Azar et al., 2024).

LLM Reasoning: Recent methods further extend the capabilities of LLMs to generate intermediate reasoning steps. The standard approach in LLM-based reasoning begins with an initial policy π instantiated by a reference LLM. The output is generated as a sequence of steps by auto-regressively predicting the next step using prompts. This idea is widely known through the **Chain-of-Thought (CoT)** (Wei et al., 2022) approach to reasoning tasks. **Self-Consistency** (Wang et al., 2022) selects the most frequent answer from multiple generations. **Tree-of-Thought (ToT)** (Yao et al., 2023) extends the CoT method by generating tree-based reasoning traces, encouraging exploration among various possible thoughts.

Another approach uses a **value function** to guide the selection of reasoning traces from a combinatorially large set. Two common approaches include **Outcome Reward Models (ORMs)** (Cobbe et al., 2021) and **Process Reward Models (PRMs)** (Li and Li, 2024). While ORMs are trained only on the accuracy of the final answer, PRMs are trained on the accuracy of intermediate reasoning steps. A popular approach, **Best-of-N** (Lightman et al., 2023), combines the utility of a value function (either PRM or ORM) by selecting the reasoning trace with the highest value.

Inference-Time Scaling: Inference-time compute methods aim to enhance the reasoning capabilities of LLMs by allowing the models to 'think' longer or perform additional computation during inference. (Snell et al., 2024) provides a unified perspective on test-time computation as modifying an LLM's distribution. Specifically, we focus on methods that modify outputs using **post-hoc scorers**, typically employed in **Markov Chain Monte Carlo (MCMC) algorithms** (Andrieu et al., 2003).

ReST (Singh et al., 2023) is a self-training method where the model generates samples filtered by binary feedback, fine-tunes on these filtered samples, and repeats the process. It outperforms human-only fine-tuning and reduces reliance on human data. **STaR (Self-Taught Reasoner)** (Zelikman et al., 2024) generates rationales using a few examples, retries with the correct answer if the output is wrong, fine-tunes on successful rationales, and repeats the process. This enhances performance on reasoning tasks without large rationale datasets, rivaling much larger models.

MCTS-based approaches and similar planning strategies (Liu et al., 2023; Zhang et al., 2023; Zhou et al., 2023; Choi et al., 2023) that leverage a learned value network have demonstrated powerful test-time scaling by guiding inference with value estimates, resulting in more coherent and preferable text generation. Tree-based methods, like **Tree-of-Thought** (Yao et al., 2023) and **RAP** (Hao et al.,

2023), enhance LLM reasoning with shallow tree-search but struggle with long-horizon tasks or limited model knowledge.

TS-LLM (Feng et al., 2023) addresses these issues using an AlphaZero-inspired framework with a learned value function, enabling deeper, more adaptable reasoning and improving LLM performance during both inference and training. **ReST-MCTS*** (Zhang et al., 2024) integrates enhancements from ReST and TS-LLM by using tree-search reinforcement learning to infer per-step rewards, enabling higher-quality reasoning trace collection and iterative policy training, outperforming prior methods like Best-of-N.

TreeBoN (Qiu et al., 2024) improves upon Best-of-N sampling by using speculative tree-search and token-level DPO rewards to prune low-quality paths, achieving better output quality with lower computational cost and outperforming standard Best-of-N across multiple benchmarks.

s1 (Muennighoff et al., 2025) is a simple test-time scaling method that improves model reasoning by using a curated reasoning dataset (s1K) and a technique called **budget forcing** to control its thinking length. This method forcefully terminates the model's output or extends it by appending a "Wait" token multiple times when the model attempts to end.

B Proofs of Base Results

In this section, we provide detailed proofs of the main and auxiliary results needed to derive the structure of the optimal function $Q^{*,\eta}, V^{*,\eta}$.

B.1 Notational Preliminaries

We now introduce several notational conventions used frequently throughout the article. For clarity and conciseness, we typically adopt the most compact notation possible, often omitting information when the meaning is clear from context.

Given a set $\mathcal{A}$, for any two elements $x, y \in \mathcal{A}$, we denote that x is preferred over y by $x \succ y$. Therefore, the indicator function that denotes preference between two elements is $\mathbf{1}_{x \succ y}$. Recall we denote the generation starting from a given context by y_x^π according to a policy π. We interchangeably use π to denote the policy, and the LLM model as well

Greedy Decoding (GD): Let π be a language model (LLM) policy, a fixed context x, and a target generation length T. The greedy decoding strategy produces an output sequence $y_{\pi;\text{GD}} = (x, a_{1:T})$, where each token a_t is deterministically selected according to

$$a_t = \arg\max \pi(\cdot \mid x, a_{1:t-1}). \tag{22}$$

We use $x \in \mathbb{R}^d$ to denote a d-dimensional vector, $M \in R^{m \times n}$ denotes a matrix of dimension $m \times n$. The semi-norm is defined as $\|u\|_{\Sigma_\mathcal{D}+\lambda I} \triangleq \sqrt{u^\top (\Sigma_\mathcal{D} + \lambda I) u}$. The d-dimensional all-ones vectors is defined as follows

$$\mathbf{1} = [1, 1, \cdots, 1]^\top \in \mathbb{R}^d$$

The logistic function is denoted by $\sigma(x) \triangleq \frac{1}{1+e^{-x}}$, and its inverse by $\Psi(t) \triangleq \frac{t}{(1-t)}$.

B.2 Optimal Soft Q and V Functions

We provide the proof of optimality for the policy structure considered in equations 8 for the KL-regularized optimization objective 7. This is a well-established result and we only provide it for completeness. With a slight abuse of notation, we denote the optimal policy by π_t^*, and the corresponding optimal soft-value functions by V_t^*, Q_t^*

The optimal solution to the KL-regularized reward maximization objective in 3 Rafailov et al. (2023) is given by

$$\pi_r^*(y \mid x) \propto \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\eta} r(x, y)\right) \tag{23}$$

We can make the following inductive argument for the optimality of the soft-value functions $V_t^{\star,\eta}, Q_t^{\star,\eta}$ starting at at given state $s_t = s$ and action $a_t = a$.

Base case: let $s_{T-1} = s$. Since $V_T^{\star,\eta}(s) = 0$, we have by definition $Q_{T-1}^{\star,\eta}(s,a) = r(s,a)$. Therefore, we have from Equation 23 the expression for the optimal policy, and subsequently the soft-value function as

$$\pi^*_{T-1}(a \mid s) \propto \pi_{\text{ref}}(a \mid s) \exp\left(\frac{1}{\eta} Q^{\star,\eta}_{T-1}(s,a)\right) \propto \pi^{*,\eta}_{T-1}(a \mid s)$$

$$\pi^*_{T-1}(a \mid s) = \pi^{*,\eta}_{T-1}(a \mid s) = \frac{\pi_{\text{ref}}(a \mid s) \exp\left(\eta^{-1} Q^{\star,\eta}_{T-1}(s,a)\right)}{\mathbb{E}_{\pi_{\text{ref}}}\left[\exp\left(\eta^{-1} Q^{\star,\eta}_{T-1}(s,a)\right)\right]}$$

$$\begin{aligned}
V^{\star,\eta}_{T-1}(s) &= \mathbb{E}_{\pi^{*,\eta}}\left[r_{T-1}(s,a) - \eta \,\mathrm{KL}(\pi^{*,\eta}(s_{T-1}) \parallel \pi_{\text{ref}}(s_{T-1}))\Big| s_{T-1} = s\right] \\
&= \mathbb{E}_{\pi^*}\left[Q^{\star,\eta}_{T-1}(s,a) - \eta \,\mathrm{KL}(\pi^*(s_{T-1}) \parallel \pi_{\text{ref}}(s_{T-1}))\Big| s_{T-1} = s\right] \\
&= \mathbb{E}_{\pi^*}\left[Q^{\star,\eta}_{T-1}(s,a)\right] - \eta\left[\mathrm{KL}(\pi^*(s) \parallel \pi_{\text{ref}}(s))\right] \\
&= \mathbb{E}_{\pi^*}\left[Q^{\star,\eta}_{T-1}(s,a)\right] - \eta\mathbb{E}_{\pi^*}\left[\eta^{-1} Q^{\star,\eta}_{T-1}(s,a)\right] + \eta \ln \mathbb{E}_{\pi_{\text{ref}}}\left[\exp\left(\eta^{-1} Q^{\star,\eta}_{T-1}(s,a)\right)\right] \\
&= \eta \ln \mathbb{E}_{\pi_{\text{ref}}}\left[\exp\left(\eta^{-1} Q^{\star,\eta}_{T-1}(s,a)\right)\right]
\end{aligned}$$

Induction Case: Let's suppose $\pi_t^{\star,\eta}$ is optimal from horizon $h+1$ until $T-1$.

$$\begin{aligned}
Q^{\star,\eta}_h(s,a) &= r_h(s,a) + \mathbb{E}_{s' \sim P(\cdot|s,a)}\left[V^{*,\eta}_{h+1}(s')\right] \\
&= r_h(s,a) + \mathbb{E}_{s' \sim P(\cdot|s,a)}\left[V^{*}_{h+1}(s')\right] = Q^{\star}_h(s,a)
\end{aligned}$$

Denoting the state at horizon h by $s_h = s$, using a policy of the form $\pi = (\pi_h, \pi^\star_{h+1}, \ldots, \pi^\star_{T-1})$ we have

$$\begin{aligned}
V^{\pi}_h(s) &= \mathbb{E}_{\pi}\left[\sum_{t \geq h} r_t(s,a) - \eta \,\mathrm{KL}(\pi(s_t) \parallel \pi_{\text{ref}}(s_t))\Big| s_h = s\right] \\
&= \mathbb{E}_{\pi_h}\left[r_h(s,a) - \eta \,\mathrm{KL}(\pi_h(s_h) \parallel \pi_{\text{ref}}(s_h)) + \mathbb{E}_{\pi^*}\left[\sum_{t > h} r_t(s,a) - \eta \,\mathrm{KL}(\pi^*(s_t) \parallel \pi_{\text{ref}}(s_t))\Big| s_h = s\right]\right] \\
&= \mathbb{E}_{\pi_h}\left[r_h(s,a) - \eta \,\mathrm{KL}(\pi_h(s_h) \parallel \pi_{\text{ref}}(s_h)) + \mathbb{E}_{s' \sim P(\cdot|s,a)}\left[V^{*}_{h+1}(s')\right]\right] \\
&= \mathbb{E}_{\pi_h}\left[Q^{\star,\eta}_h(s,a) - \eta \,\mathrm{KL}(\pi_h(s_h) \parallel \pi_{\text{ref}}(s_h))\right]
\end{aligned}$$

Using the results of Rafailov et al. (2023) by setting $r(s,a) = Q^\star_h(s,a)$,

$$V^\star_h(s) = \max_{\pi_h} \mathbb{E}_{\pi_h}\left[Q^\star_h(s,a) - \eta \,\mathrm{KL}(\pi(s_h) \parallel \pi_{\text{ref}}(s_h))\right] = V^{\star,\eta}_h(s)$$

For deterministic MDP's, the expression for the optimal soft-value expressions has a simple form in terms of the cumulative rewards from rollouts according to π_{ref} Zhou et al. (2025) as shown below.

$$\begin{aligned}
\exp(\eta^{-1} V^{\star,\eta}_t(s)) &= \mathbb{E}_{\pi^{\text{ref}}}\left[\exp\left(\eta^{-1} r_t(s_t,a_t) + \eta^{-1} V^{\star,\eta}_{t+1}(s_{t+1})\right)\right] \\
&= \mathbb{E}_{\pi^{\text{ref}}}\left[\exp\left(\eta^{-1} \sum_t r_t(s_t,a_t)\right)\Bigg| s_t = s\right]
\end{aligned}$$

Taking logarithm on both sides, we get

$$V^{\star,\eta}_t(s) = \eta \ln \mathbb{E}_{\pi^{\text{ref}}}\left[\exp\left(\eta^{-1} \sum_t r_t(s_t,a_t)\right)\Bigg| s_t = s\right]$$

The result for $Q^{\star,\eta}_t$ follows a similar line of analysis.

B.3 Proof of Lemma 1

Proof. We use a result on the equivalence class induced by two reward functions under the same preference distribution from Rafailov et al. (2023). For any given context x, and policy π, let y_x^π be the unique greedy completion under policy π. Setting $\tilde{r}(y_x) \triangleq r^*(y_x) - r^*(y_x^\pi)$, we observe that $\tilde{r}(y_x) - r^*(y_x)$ is entirely a functional of x. It follows from Rafailov et al. (2023)[Lemma 1] that the preference distribution induced by $\tilde{r}(\cdot)$ is identical to $\mathcal{P}^*$. □

B.4 Proof of Theorem 1

Proof. The inverse function of the logistic function $\sigma(x)$ is given by $\Psi(x) \triangleq \log \frac{x}{(1-x)}$. Suppose that the Bradley-Terry model holds. For any y, y' given a context x, we have:

$$\begin{aligned}\Psi\left(\mathcal{P}^*(y \succ y'|x)\right) &= \Psi(\sigma(r(y) - r(y'))) \\ &= r(y) - r(y') \\ r(y) &= r(y') + \Psi\left(\mathcal{P}^*(y \succ y'|x)\right).\end{aligned}$$

Setting $y := s_T$, and $y' := y_s^{\text{ref}}$, the above equivalence of the reward function to the preference distribution yields the following expressions for the optimal value function $V_t^{\star,\eta}$

$$\begin{aligned}V_t^{\star,\eta}(s) &= \eta \ln \mathbb{E}_{\pi_{\text{ref}}}\left[\exp\left(\eta^{-1} r(s_{T-1}, a_{T-1})\right) \middle| s_t = s\right] && (24)\\ &= \eta \ln \mathbb{E}_{y_s \sim \pi_{\text{ref}}}\left[\exp\left(\eta^{-1}\left[r(y_s^{\text{ref}}) + \Psi\left(\mathcal{P}^*(y_s \succ y_s^{\text{ref}})\right)\right]\right) \middle| s_t = s\right], && (25)\\ &= r(y_s^{\text{ref}}) + \eta \ln \mathbb{E}_{y_s \sim \pi_{\text{ref}}}\left[\exp\left(\eta^{-1}\left[\Psi\left(\mathcal{P}^*(y_s \succ y_s^{\text{ref}})\right)\right]\right) \middle| s_t = s\right], && (26)\\ & && (27)\end{aligned}$$

Similarly, the expression of $Q_t^{\star,\eta}$ can be written in terms of the preference distribution. □

C Convergence of PITA

C.1 Bounding the MLE estimation error

We present a proof sketch of the MLE estimation error bound in Lemma 2, which closely follows Zhu et al. (2023) [Lemma 3.1]. We first establish the strong convexity of the loss function, and then bound the estimation error to obtain the final result.

Definition 1. *(Strong Convexity) (Beck, 2017)[Theorem 5.8] A function $f : \mathbb{E} \to (-\infty, \infty]$ is called σ-strongly convex for a given $\sigma > 0$ if $\text{dom}(f)$ is convex and the following inequality holds for any $\mathbf{x}, \mathbf{y} \in \text{dom}(f)$ and $\lambda \in [0, 1]$:*

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \langle \mathbf{g}, \mathbf{y} - \mathbf{x} \rangle + \frac{\sigma}{2}\|\mathbf{x} - \mathbf{y}\|^2 \quad \textit{for any } \mathbf{x} \in \text{dom}(\partial f),\ \mathbf{y} \in \text{dom}(f) \textit{ and } \mathbf{g} \in \partial f(\mathbf{x}),$$

where $\partial f(\mathbf{x})$ is the sub-differential of f at $\mathbf{x}$.

Lemma 3 (Strong Convexity of the MLE Loss)**.** *The MLE loss $\ell_{\mathcal{D}}$ is strongly convex at $\theta^\star$ with respect to the semi-norm $\|\cdot\|_{\Sigma_{\mathcal{D}}}$. Specifically, there exists a constant $\gamma > 0$ such that*

$$\ell_{\mathcal{D}}(\theta^\star + \Delta) - \ell_{\mathcal{D}}(\theta^\star) - \langle \nabla \ell_{\mathcal{D}}(\theta^\star), \Delta \rangle \geq \gamma \|\Delta\|_{\Sigma_{\mathcal{D}}}^2 \tag{28}$$

for all perturbations $\Delta \in \mathbb{R}^d$ such that $\theta^\star + \Delta \in \Theta_B$.

Proof. For simplicity, we define $x_i \triangleq \Phi(y_i)$. The explicit expression for $\ell_{\mathcal{D}}$ is given by:

$$\ell_{\mathcal{D}}(\theta) = -\sum_{i=1}^{n} o_i \log \sigma(\langle \theta, x_i \rangle) + (1 - o_i) \cdot \log\left(1 - \sigma(\langle \theta, x_i \rangle)\right), \tag{29}$$

$$\tag{30}$$

where $o_i = \mathbf{1}_{y_i \succ y_i^{\text{ref}}}$.

Next, we compute the Hessian of ℓ:

$$\nabla^2 \ell_{\mathcal{D}}(\theta) = \frac{1}{n}\sum_{i=1}^{n}\left(o_i \cdot \frac{\exp(-\langle\theta, x_i\rangle)}{(\exp(-\langle\theta, x_i\rangle)+1)^2} + (1-o_i)\cdot\frac{\exp(\langle\theta, x_i\rangle)}{(\exp(\langle\theta, x_i\rangle)+1)^2}\right)\cdot x_i x_i^\top$$
$$= \frac{1}{n}\sum_{i=1}^{n}\frac{\exp(-\langle\theta, x_i\rangle)}{(\exp(-\langle\theta, x_i\rangle)+1)^2}\cdot x_i x_i^\top.$$

From Assumption 1, we have that $\langle\theta, x_i\rangle \in [-2LB, 2LB]$. This gives the following lower bound:

$$\frac{\exp(-\langle\theta, x_i\rangle)}{(\exp(-\langle\theta, x_i\rangle)+1)^2} \geq \frac{1}{2+\exp(-2LB)+\exp(2LB)}.$$

Using this result and the expression for the Hessian above, we find:

$$z^\top \nabla^2 \ell_{\mathcal{D}}(\theta) z \geq \frac{\gamma}{n}\|Xz\|_2^2 \quad \text{for all } z,$$

where $\gamma \triangleq \frac{1}{2+\exp(-2LB)+\exp(2LB)}$ and $X \in \mathbb{R}^{n\times d}$, with the i^{th} row of X given by $x_i \in \mathbb{R}^d$.

Finally, using the second-order mean value theorem, we obtain:

$$\ell_{\mathcal{D}}(\theta^\star+\Delta) - \ell_{\mathcal{D}}(\theta^\star) - \langle\nabla\ell_{\mathcal{D}}(\theta^\star), \Delta\rangle \geq \gamma\|\Delta\|_{\Sigma_{\mathcal{D}}}^2,$$

where the semi-norm is defined as $\|u\|_{\Sigma_{\mathcal{D}}+\lambda I} \triangleq \sqrt{u^\top(\Sigma_{\mathcal{D}}+\lambda I)u}$. □

Proof sketch of Lemma 2. By definition $\hat{\theta}_{\text{MLE}}$ is optimal for ℓ_D, hence $\ell_D(\hat{\theta}_{\text{MLE}}) \leq \ell_D(\theta^\star)$. Setting the error vector $\Delta = \hat{\theta}_{\text{MLE}} - \theta^\star$, from the γ-convexity condition, and Holder's inequality on the norm $\|\cdot\|_{\Sigma_{\mathcal{D}}+\lambda I}$ it follows that

$$\gamma\|\Delta\|_{\Sigma_D}^2 \leq \ell_D(\theta^\star+\Delta) - \ell_D(\theta^\star) - \langle\nabla\ell_D(\theta^\star), \Delta\rangle$$
$$\leq -\langle\nabla\ell_D(\theta^\star), \Delta\rangle \leq \|\nabla\ell_D(\theta^\star)\|_{(\Sigma_D+\lambda I)^{-1}}\|\Delta\|_{\Sigma_D+\lambda I}.$$

With probability at least $(1-\delta)$, the term $\|\nabla\ell_D(\theta^\star)\|_{(\Sigma_D+\lambda I)^{-1}}$ is further bounded using tail concentration inequalities Hsu et al. (2012) as follows

$$\|\nabla\ell_{\mathcal{D}}(\theta^*)\|_{(\Sigma_{\mathcal{D}}+\lambda I)^{-1}}^2 \leq \tilde{C}\cdot\frac{d+\log(1/\delta)}{n}.$$

for some universal constant C. Assumption 1, combined with previous results gives us

$$\gamma\|\Delta\|_{\Sigma_{\mathcal{D}}+\lambda I}^2 \leq \|\nabla\ell_{\mathcal{D}}(\theta^*)\|_{(\Sigma_{\mathcal{D}}+\lambda I)^{-1}}\|\Delta\|_{\Sigma_{\mathcal{D}}+\lambda I} + 4\lambda\gamma B^2 \tag{31}$$
$$\leq \sqrt{\tilde{C}\cdot\frac{d+\log(1/\delta)}{n}}\|\Delta\|_{\Sigma_{\mathcal{D}}+\lambda I} + 4\lambda\gamma B^2. \tag{32}$$
$$\|\Delta\|_{\Sigma_{\mathcal{D}}+\lambda I} \leq C\cdot\sqrt{\frac{d+\log(1/\delta)}{\gamma^2 n} + \lambda B^2}. \tag{33}$$

Where C can be chosen to be an $o(1)$ constant which is bounded by a multiple of $\sqrt{\tilde{C}}$.

□

C.2 Proof of Theorem 2

Before delving into the details, we provide a brief overview of the steps involved in deriving the regret bound on PITA. In contrast to the approach of Zhou et al. (2025), we do not directly rely on the Performance Difference Lemma. Instead, we leverage a closed-form expression for $V^{\theta,\eta}(\cdot)$ that explicitly depends on the parameter θ. Our key insight is that the gradient of this parameterized value function with respect to θ can be expressed directly in terms of the policy space; see Equation equation 43.

Within this framework, Assumption 2 plays a crucial role by bounding the quantity $\mathbb{E}_{y_x\sim\pi_\theta(y_x)}\left[\eta^{-1}\Phi(y_x)\right]$ for any θ. This condition is essential because it excludes deterministic policies and ensures a degree of exploration across all learned policies. Notably, this assumption aligns with standard practices in KL-regularized policy updates, where a similar condition is imposed with π_θ replaced by a reference policy π_{ref}. In our case, this reference policy corresponds to an initial estimate of θ.

Proof. For a given iteration k, let $\theta_k = \hat{\theta}$ be the MLE estimate. Consider the following set of parameters:

$$\Theta(\hat{\theta},\lambda) = \left\{ \theta \in \Theta_B \;\middle|\; \left\|\hat{\theta}-\theta\right\|_{\Sigma_{\mathcal{D}}+\lambda I} \le C\cdot\sqrt{\frac{d+\log\left(\frac{1}{\delta}\right)}{\gamma^2 n}+\lambda B^2} \right\}. \tag{34}$$

For a given context x, the difference between the optimal value function and the estimated value function is as follows:

$$V^{\star,\eta}(x) - V^{\hat{\theta},\eta}(x) = \eta \ln \mathbb{E}_{y_x\sim\pi_{\text{ref}}}\left[\exp\left(\eta^{-1}\langle\theta^*,\Phi(y_x)\rangle\right)\middle|x\right] \tag{35}$$

$$- \eta \ln \mathbb{E}_{y_x\sim\pi_{\text{ref}}}\left[\exp\left(\eta^{-1}\langle\hat{\theta},\Phi(y_x)\rangle\right)\middle|x\right] \tag{36}$$

By the mean value theorem, we have for some $\bar{\theta}$ on the line joining θ^* and $\hat{\theta}$ that:

$$\left|\ln \mathbb{E}_{y_x\sim\pi_{\text{ref}}}\left[\exp\left(\eta^{-1}\langle\theta^*,\Phi(y_x)\rangle\right)\middle|x\right] - \ln \mathbb{E}_{y_x\sim\pi_{\text{ref}}}\left[\exp\left(\eta^{-1}\langle\hat{\theta},\Phi(y_x)\rangle\right)\middle|x\right]\right| \tag{37}$$

$$= \left|\langle\theta^*-\hat{\theta}, \nabla_\theta \ln \mathbb{E}_{y_x\sim\pi_{\text{ref}}}\left[\exp\left(\eta^{-1}\langle\theta,\Phi(y_x)\rangle\right)\middle|x\right]\mid_{\theta=\bar{\theta}}\rangle\right| \tag{38}$$

From Hölder's inequality on the $\|\cdot\|_{\Sigma_{\mathcal{D}}+\lambda I}$ norm and its dual, we obtain:

$$\left|\langle\theta^*-\hat{\theta}, \nabla_\theta \ln \mathbb{E}_{y_x\sim\pi_{\text{ref}}}\left[\exp\left(\eta^{-1}\langle\theta,\Phi(y_x)\rangle\right)\middle|x\right]\mid_{\theta=\bar{\theta}}\rangle\right| \tag{39}$$

$$\le \|\hat{\theta}-\theta\|_{\Sigma_{\mathcal{D}}+\lambda I}\|\nabla_\theta \ln \mathbb{E}_{y_x\sim\pi_{\text{ref}}}\left[\exp\left(\eta^{-1}\langle\theta,\Phi(y_x)\rangle\right)\middle|x\right]\mid_{\theta=\bar{\theta}}\|_{(\Sigma_{\mathcal{D}}+\lambda I)^{-1}} \tag{40}$$

$$= \left\|(\hat{\theta}-\theta)(\Sigma_{\mathcal{D}}+\lambda I)^{1/2}\right\|_2 \left\|(\Sigma_{\mathcal{D}}+\lambda I)^{-1/2}\nabla_\theta \ln \mathbb{E}_{y_x\sim\pi_{\text{ref}}}\left[\exp\left(\eta^{-1}\langle\theta,\Phi(y_x)\rangle\right)\middle|x\right]\mid_{\theta=\bar{\theta}}\right\|_2 \tag{41}$$

$$= \left\|(\hat{\theta}-\theta)\right\|_{\Sigma_{\mathcal{D}}+\lambda I} \left\|(\Sigma_{\mathcal{D}}+\lambda I)^{-1/2}\nabla_\theta \ln \mathbb{E}_{y_x\sim\pi_{\text{ref}}}\left[\exp\left(\eta^{-1}\langle\theta,\Phi(y_x)\rangle\right)\middle|x\right]\mid_{\theta=\bar{\theta}}\right\|_2 \tag{42}$$

The gradient of the log-expectation in the second term can be written as the expected embedding under policy $\pi_{\bar{\theta}} \in \Pi_\theta$. That is:

$$\nabla_\theta \ln \mathbb{E}_{y_x\sim\pi_{\text{ref}}}\left[\exp\left(\eta^{-1}\langle\theta,\Phi(y_x)\rangle\right)\middle|x\right]_{\theta=\bar{\theta}} = \frac{1}{\eta}\mathbb{E}_{y_x\sim\pi_{\bar{\theta}}(y_x)}\left[\Phi(y_x)\right] \tag{43}$$

Taking the expectation with respect to the distribution of context x and summing over K iterations, from Assumption 2, we obtain the following result:

$$\sum_{k=1}^{K}(V^{\star,\eta}-V^{\theta_k,\eta}) \le C\sum_{k=1}^{K}\left(\sqrt{\frac{d+\log(1/\delta)}{\gamma^2 n_k}+\lambda B^2}\cdot\sup\left\|(\Sigma_{\mathcal{D}}+\lambda I)^{-1/2}\mathbb{E}_{y\sim\pi_{\text{ref}}}[\Phi(y)]\right\|_2\right). \tag{44}$$

□

D Math Reasoning, Star Graph Reasoning, and Sentiment Generation

In this section we present our results for sentiment generation tasks Chaudhary et al. (2025), star graph reasoning, and provide the training details for PITA on GSM8K.

D.1 MATH REASONING

Dataset. **GSM8K** contains $7{,}500$ training and $1{,}319$ test problems. We create a $90\%/10\%$ split of the original training set for learning and validation, reserving the full test set for final reporting.

Models. We use the **Llama-3 8B-Instruct** model as our reference policy and the **Llama-3.2 1B-Instruct** model Grattafiori et al. (2024) as the backbone for our Value function estimation. For Q#-HF, the reward model trained from preference data is also a Llama-3.2 1B-Instruct. We use the OpenMathInstruct-2 model Toshniwal et al. (2024) for preference data generation between two pairs of responses for it's reasonably high accuracy on arithmetic reasoning tasks such as GSM8K.

Training details. We collect pairwise samples for each question in the training set of GSM8K and label every sample either as *'preferred'* (1) or *'not preferred'* (0) based on the quality of final answer. Note that for arithmetic reasoning, there is an exact numeric answer for each question. A correct answer is automatically preferred over a wrong answer. If the samples from a given pair are either both correct, or both wrong a preference is assigned based by the OpenMathInstruct-2 model. PITA is trained over 10 policy evaluation/optimization cycles (5 per round), using a learning rate of 2×10^{-5} and a context length of 4096. The model is retrained from scratch at the start of each round, using data collected from both the current and all previous rounds. The baseline model is trained with identical hyperparameters. Each optimization run is performed on a single NVIDIA A100 GPU.

Evaluation protocol. We report single-sample accuracy (*pass@1*) and majority-vote accuracy over k=8 samples (*maj1@8*). Generations use temperature $T = 0.8$ and nucleus sampling $p = 0.9$. We use the inference optimizations for value function V given in Zhou et al. (2025). Similar to Q# we train PITA for two iterations and observe performance converging. All evaluations are done with zero-shot prompting.

The prompt template used for evaluation is: `'Problem:\n\n{0} Write your answer inside \\boxed{{}}.\n\nSolution:'` where `{0}` is replaced by the actual question from the dataset.

D.2 STAR-GRAPH REASONING

To better understand the effectiveness of our method in guiding a reference model at test time, we analyze the behavior of PITA alongside other value-guided algorithms on star-graph reasoning tasks (Bachmann and Nagarajan, 2024).

A star-graph $\mathcal{G}(d, l)$ is defined as a tree with d paths of length l emanating from a common root node. The task is for the language model (LM) to generate the correct path from the root to a leaf, given the edge set defining the graph. Figure 4 illustrates the star-graph task.

In addition to post-training algorithms, we compare the performance of PITA with the following methods:

- **CD** Mudgal et al. (2023): A controlled decoding algorithm trained with access to true binary reward labels.
- **REINFORCE** (Ahmadian et al., 2024): A post-training algorithm that uses REINFORCE instead of the canonical Proximal Policy Optimization (PPO) in reinforcement learning from human feedback.
- **DPO** (Rafailov et al., 2023): A post-training algorithm that aligns language models with human preferences by optimizing for preferred responses over less preferred ones, without using reinforcement learning.
- **RPO** (Pang et al., 2024): A post-training algorithm that modifies the DPO loss to improve performance on reasoning tasks.

Despite the task's structural simplicity, Bachmann and Nagarajan (2024) highlight a key failure mode in models trained via next-token prediction. Specifically, the model often learns a *Clever-Hans* shortcut—selecting the first node in the root's adjacency list and following it to a leaf. While this heuristic yields perfect accuracy on the training set, it fails to generalize: on unseen test graphs, the model selects the correct path only with probability $1/d$, as the first adjacent node is correct only $1/d$ of the time.

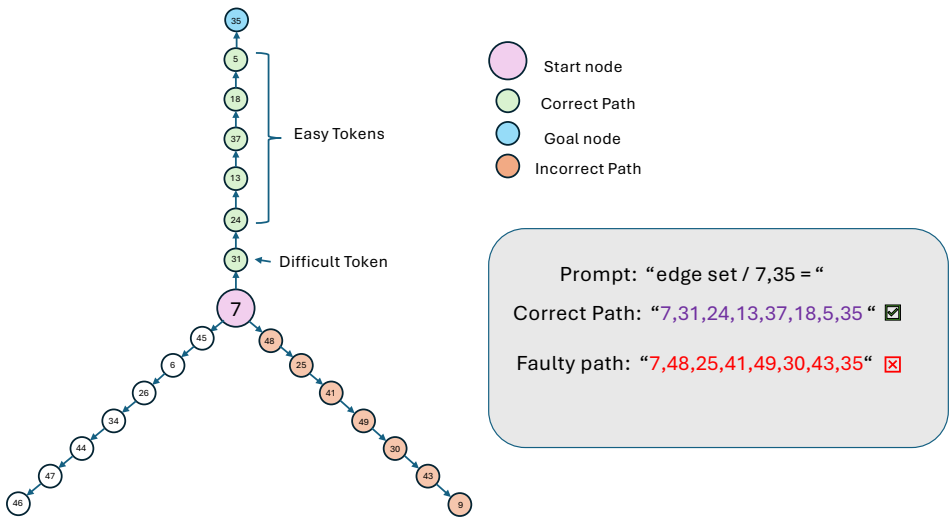

Figure 4: A $\mathcal{G} = (3, 8)$ star-graph configuration with degree $d = 3$ and path length $l = 8$. The pre-trained model, using next-token prediction, learns a faulty shortcut: it selects a random first node and follows the path from there.

Zhou et al. (2025) show that this faulty behavior can be corrected during post-training using value-based methods. However, widely used policy-based approaches such as REINFORCE, DPO, and RPO fail to overcome this shortcut, achieving test accuracy no better than $1/d$. This limitation is likely due to the difficulty transformer-based architectures face in unlearning spurious heuristics via policy-gradient optimization (Hu et al., 2024).

Main Results: In Table 3, we report the test accuracies of PITA on three classes of graphs: smaller graphs $\mathcal{G}(2, 5)$ and $\mathcal{G}(5, 5)$, where we use the GPT-2 small model, and a larger graph $\mathcal{G}(3, 8)$, where we use the GPT-2 medium model Radford et al. (2019) to train the set of pre-trained models. We observe that **PITA** effectively guides the reference LM to achieve near-perfect generalization accuracy on the star-graph task, outperforming strong value-based baselines such as Q# and CD, which themselves achieve high generalization performance.

Table 3: Comparison of PITA with other baselines across star-graph configurations.

Algorithm	$\mathcal{G}(2, 5)$	$\mathcal{G}(5, 5)$	$\mathcal{G}(3, 8)$
pre-trained	49.94	19.8	33.24
Q#	99.90	97.00	98.62
CD	99.94	98.62	99.52
REINFORCE	49.19	13.13	25.96
DPO	37.61	0.00	0.00
RPO	49.69	19.93	33.47
PITA (ours)	99.97	99.70	99.93

The star-graph experiments provide insight into why PITA outperforms standard policy-based methods, as well as value-based algorithms like Q# and CD. While REINFORCE achieves a maximum accuracy of $1/d$, DPO performs even worse. We observe behavior similar to that reported in Zhou et al. (2025), where DPO achieves 0 accuracy due to policy collapse (i.e., assigning zero probability mass). In contrast, value-based methods perform well across all graph configurations. Notably, PITA further improves accuracy, even when baseline value-based methods already achieve near-perfect performance.

This improvement can be attributed to PITA's ability to generate more precise implicit rewards. Upon examining the rewards generated by Q# on the star-graph reasoning test set, we find that the reward differences between correct and incorrect answers are significantly smaller than those produced by PITA.

We now describe the training methodology used to apply **PITA** to the star-graph task.

Training Setup: We follow the experimental setup of Q# Zhou et al. (2025) to generate and evaluate the star-graph datasets. All models are initially pretrained using next-token prediction on a dataset of 200k randomly sampled graphs paired with their correct paths.

We consider several baselines, including **REINFORCE**, **DPO**, **RPO**—as well as value-based methods **Q#** and **CD**. Post-training is conducted on a separate dataset of 200k new random graphs. To ensure a fair comparison, we reproduce the results with an identical reward structure and reuse the datasets used in Zhou et al. (2025). For REINFORCE, a reward of 1 is assigned for correct paths and -0.1 for incorrect ones. For DPO and RPO, pairwise comparisons between a correct path y_{chosen} and an incorrect shortcut path y_{reject} are sampled from the pre-trained model. Similarly, for **Q#**, a classifier is trained on the same pairwise dataset, labeling correct paths with reward 1 and incorrect ones with 0. Note that PITA uses a unique reference generation at the core of its algorithm. In PITA, we treat correct paths as the *'preferred'* solution and the incorrect path as the *'not-preferred'* answer to estimate the preference distribution. The unique correct answer naturally serves as the reference completion for the star-graph task.

All models are trained for 10 epochs using the AdamW optimizer with a weight decay of 0.1 and a batch size of 256. The learning rates are set to 2.5×10^{-4} for pretraining, 1×10^{-5} for REINFORCE, and 1×10^{-4} for DPO, RPO, CD, and Q#. For Q# and CD, we set the inverse temperature parameter $\eta = 0.1$. Training is performed on a single A100 GPU. Evaluation is conducted on a held-out test set of 20k graphs using top-$k = 10$ sampling and temperature 1.0.

D.3 IMDB Sentiment Generation

In this section we evaluate **PITA** on controlled sentiment generation. The task follows the IMDB-Gen setting of Ramamurthy et al. (2023) and Chaudhary et al. (2025). Given the first 60 tokens of a movie review, the language model must continue the text so that the completed review expresses maximally positive sentiment.

Dataset. We use the **IMDB Reviews** dataset, which contains $25{,}000$ training and $25{,}000$ test reviews, each labeled *positive* or *negative*. For each example, we truncate the review to its first 60 tokens, which form the prompt x; the model then generates up to 60 continuation tokens y. The reward model for this task is the **lvwerra/distilbert-imdb** classifier, with the logit corresponding to the positive sentiment output node used as the scalar reward. For the creation of the preference dataset, we select the generation with the higher reward as the preferred generation.

Models. For this task, we use **TinyLlama-1.1B-Chat**[1] as both our reference policy and the backbone for our value function. For the reward model learned from the preference data for Q#-HF, we use a **LLama 3.2 1B-Instruct** model.

Evaluation We evaluate the same set of algorithms as above. For each of the $5{,}000$ **IMDB** test set prompts, we sample a *single* continuation using nucleus sampling with top-$p = 0.9$ and temperature $T = 0.8$. The full prompt + completion is scored by the `lvwerra/distilbert-imdb` sentiment classifier. We report the mean sentiment logit score (SENT.↑) and the token-level KL divergence (KL↓) between the fine-tuned policy and the reference model π_{ref}. An example of the completions generated by π_{ref} and by PITA is shown in Figure 5.

[1] `https://huggingface.co/TinyLlama/TinyLlama-1.1B-Chat`

Table 4: IMDB-Gen: sentiment (higher is better) and KL divergence (lower is better) on 5,000 test prompts.

Algorithm	SENT.↑	KL↓
π_{ref}	-0.52	-
Q#	0.22	7.51
Q#-HF	-0.44	5.22
PITA (ours)	0.01	7.27

Discussion. The range of rewards generated by `lvwerra/distilbert-imdb` typically lies between -3 and +3. Q# with access to true rewards performs the best. Among the algorithms that only have access to a preference signal, rather than the true reward, PITA performs the best.

Sentiment alignment is inherently difficult, and Q# has access to a pre-trained DistilBERT model specifically tuned for sentiment analysis. In contrast, reproducing a preference function with a similar structure requires much more preference data. Despite this challenge, PITA is able to extract valuable information from the given preference data and significantly outperforms Q#-HF, which appears to have poorly learned a reward model. This demonstrates PITA's ability to leverage limited preference data for effective alignment.

Prompt: Paul Mazursky misfires on this film. The writing, direction, casting, and acting (with the exception of Victorio Gassman) are all off the mark. I remember the reviews from 20+ years ago being mediocre, but I thought it still might

---

Completion from π^{ref}: be good. But by today's standards, the film just isn't good enough. It's a muddled and overly long (about 2 1/2 hours) film that ultimately does not live up to its potential as a classic film noir.

Completion from PITA: be good. I'm glad to see how far it's come. This is not a great film, but it's definitely a lot better than the original. I think that makes it more enjoyable to watch. The acting is excellent, with a lot of great performances by both Mar

Figure 5: Comparison of IMDB review completions (constrained to 60 tokens) from π_{ref} vs PITA. PITA is able to change the sentiment positive, while π_{ref} fails to do so.

E ADDITIONAL EXPERIMENTAL DETAILS

E.1 ABLATION STUDY ON PITA TRAINING FOR GSM8K

We benchmark **PITA** on the GSM8K grade-school arithmetic corpus Cobbe et al. (2021), which was also considered in the main paper. In contrast to the main paper, we evaluate PITA with only a single round of training i.e. ($k = 1$ in Algorithm 1). We present our results in Table 5

Table 5: Performance on the GSM8K *test* split.

Algorithm	**pass@1** ↑	**maj@8** ↑	**KL**
π_{ref}	69	85	-
Q#	78.4	88.1	2.65
Q#-HF	49.9	72.0	18.85
PITA (ours)	74.8	86.1	18.4

We see a clear drop in performance when trained with limited data in both pass@1, and maj@8 metrics. Although the choice of two rounds ($k = 2$ in Algorithm 1) was made to achieve a fairer comparison with Q#, an additional round of training allows for a greater diversity of preference data. The iterative training process is a key step in enabling PITA to better adapt to the preferences, leading to more robust results in alignment tasks.

E.2 Reward Training for Q#-HF

Recall that Q#-HF trains a reward model as an intermediate step for using the Q# algorithm. We would like to highlight the sensitivity of algorithmic performance of reward training methods. The results from Table 5 involve a single round of training. That is, to ensure a fair comparison with other algorithms, the reward function for single-round Q#-HF was trained using only 50% of the data used in the main paper, so that the total data across reward estimation and policy training remained comparable. We observe a substantial drop in performance of the Q# baseline which can we attributed to the poor reward separation between correct and incorrect generations as shown below.

Reward-Model Calibration: Figure 6 contrasts reward scores assigned by the BT model to correct versus incorrect generations produced by the baseline. A larger separation indicates stronger alignment between the reward and task success.

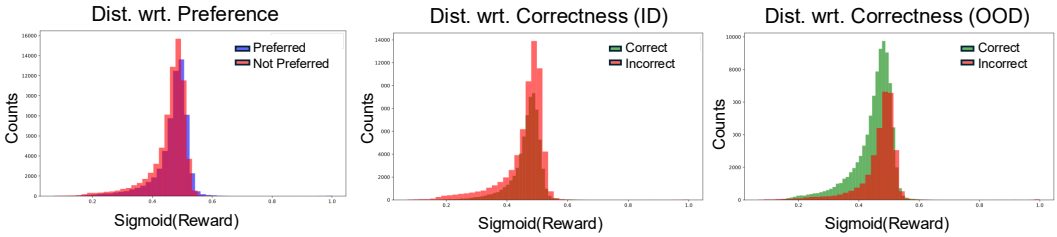

Figure 6: Distribution of learned reward scores w.r.t preferred/correct (green) and not preferred/incorrect (red) for ID(in-dist.)/OOD(out-of-dist.) data. The reward function is difficult to learn.

E.3 Comparison of PITA and Standard RLHF Training and Inference Costs

In this section, we make a concise comparison of computational costs of our inference time algorithm and standard fine-tuning approaches like RLHF/DPO. We observe that the proposed PITA approach incurs less than half the training time of standard preference-optimization methods such as DPO (even when both are applied to the same 1B model), while maintaining comparable inference-time compute.

Training Requirements for PITA vs RLHF/DPO Unlike RLHF or DPO, PITA focuses exclusively on inference-time alignment and does not require fine-tuning of the large base model. Direct comparisons across these two techniques are therefore not strictly equivalent, since RLHF/DPO typically needs to fine tune the larger base model, whereas PITA only fine tunes a smaller guidance model. Nevertheless, for the sake of comparison we report training times for both DPO and PITA on a LLaMA 1B model, using a single NVIDIA A100 GPU and the same training dataset.

- PITA (LLaMA 1B): Training required 5,494 seconds (1.53 GPU-hours) over 19,875 steps, with each step processing 4,096 tokens, totaling 81.4M tokens.
- DPO with LoRA (LLaMA 1B): Training required 13,159 seconds (3.66 GPU-hours) over 44,173 steps, with each step processing 2,048 tokens, totaling 90.5M tokens.

This gap would widen even further for larger base models, where DPO's requirement to fine-tune the model, despite using LoRA, would result in significantly higher computational cost compared to PITA.

Inference Time. Inference was benchmarked on the GSM8K test set (1,319 examples) using a single NVIDIA A100 GPU. For RLHF/DPO-style methods, inference corresponds to a single forward

pass through a LLaMA 8B model. For PITA, inference involves a forward pass through the same LLaMA 8B base model, followed by guidance from a LLaMA 1B model that modifies logits in real time. We see PITA achieves its improvements with only minimal extra compute overhead."

- PITA inference: Estimated compute cost: 2.91×10^{17} FLOPs.
- Reference model inference (RLHF/DPO): Estimated compute cost: 2.60×10^{17} FLOPs.

REFERENCES

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms. *arXiv preprint arXiv:2402.14740*, 2024.

Riad Akrour, Marc Schoenauer, and Michèle Sebag. April: Active preference learning-based reinforcement learning. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2012, Bristol, UK, September 24-28, 2012. Proceedings, Part II 23*, pages 116–131. Springer, 2012.

Christophe Andrieu, Nando De Freitas, Arnaud Doucet, and Michael I Jordan. An introduction to MCMC for machine learning. *Machine learning*, 50:5–43, 2003.

Peter Auer, Alexander Clark, Thomas Zeugmann, and Sandra Zilles. *Algorithmic Learning Theory: 25th International Conference, ALT 2014, Bled, Slovenia, October 8-10, 2014, Proceedings*, volume 8776. Springer, 2014.

Mohammad Gheshlaghi Azar, Zhaohan Daniel Guo, Bilal Piot, Remi Munos, Mark Rowland, Michal Valko, and Daniele Calandriello. A general theoretical paradigm to understand learning from human preferences. In *International Conference on Artificial Intelligence and Statistics*, pages 4447–4455. PMLR, 2024.

Gregor Bachmann and Vaishnavh Nagarajan. The pitfalls of next-token prediction. *arXiv preprint arXiv:2403.06963*, 2024.

Amir Beck. *First-order methods in optimization*. SIAM, 2017.

Marc G Bellemare, Will Dabney, and Mark Rowland. *Distributional reinforcement learning*. MIT Press, 2023.

Ralph Allan Bradley and Milton E Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712*, 2023.

Christopher P Chambers, Federico Echenique, and Nicolas S Lambert. Recovering preferences from finite data. *Econometrica*, 89(4):1633–1664, 2021.

Sapana Chaudhary, Ujwal Dinesha, Dileep Kalathil, and Srinivas Shakkottai. Risk-averse finetuning of large language models, 2025. URL `https://arxiv.org/abs/2501.06911`.

Sehyun Choi, Tianqing Fang, Zhaowei Wang, and Yangqiu Song. Kcts: knowledge-constrained tree search decoding with token-level hallucination detection. *arXiv preprint arXiv:2310.09044*, 2023.

Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

Louis Faury, Marc Abeille, Clément Calauzènes, and Olivier Fercoq. Improved optimistic algorithms for logistic bandits. In *International Conference on Machine Learning*, pages 3052–3060. PMLR, 2020.

Xidong Feng, Ziyu Wan, Muning Wen, Stephen Marcus McAleer, Ying Wen, Weinan Zhang, and Jun Wang. Alphazero-like tree-search can guide large language model decoding and training. *arXiv preprint arXiv:2309.17179*, 2023.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Seungwook Han, Idan Shenfeld, Akash Srivastava, Yoon Kim, and Pulkit Agrawal. Value augmented sampling for language model alignment and personalization. *arXiv preprint arXiv:2405.06639*, 2024.

Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. Reasoning with language model is planning with world model. *arXiv preprint arXiv:2305.14992*, 2023.

Reinhard Heckel, Max Simchowitz, Kannan Ramchandran, and Martin Wainwright. Approximate ranking from pairwise comparisons. In *International Conference on Artificial Intelligence and Statistics*, pages 1057–1066. PMLR, 2018.

Joey Hejna, Rafael Rafailov, Harshit Sikchi, Chelsea Finn, Scott Niekum, W Bradley Knox, and Dorsa Sadigh. Contrastive preference learning: learning from human feedback without rl. *arXiv preprint arXiv:2310.13639*, 2023.

Daniel Hsu, Sham Kakade, and Tong Zhang. A tail inequality for quadratic forms of subgaussian random vectors. *https://arxiv.org/abs/1110.2842*, 2012.

Edward S Hu, Kwangjun Ahn, Qinghua Liu, Haoran Xu, Manan Tomar, Ada Langford, Dinesh Jayaraman, Alex Lamb, and John Langford. Learning to achieve goals with belief state transformers. *arXiv preprint arXiv:2410.23506*, 2024.

W Bradley Knox, Stephane Hatgis-Kessell, Serena Booth, Scott Niekum, Peter Stone, and Alessandro Allievi. Models of human preference for learning reward functions. *arXiv preprint arXiv:2206.02231*, 2022.

Wendi Li and Yixuan Li. Process reward model with q-value rankings. *arXiv preprint arXiv:2410.11287*, 2024.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*, 2023.

Jiacheng Liu, Andrew Cohen, Ramakanth Pasunuru, Yejin Choi, Hannaneh Hajishirzi, and Asli Celikyilmaz. Don't throw away your value model! generating more preferable text with value-guided monte-carlo tree search decoding. *arXiv preprint arXiv:2309.15028*, 2023.

Sidharth Mudgal, Jong Lee, Harish Ganapathy, YaGuang Li, Tao Wang, Yanping Huang, Zhifeng Chen, Heng-Tze Cheng, Michael Collins, Trevor Strohman, et al. Controlled decoding from language models. *arXiv preprint arXiv:2310.17022*, 2023.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35: 27730–27744, 2022.

Aldo Pacchiano, Aadirupa Saha, and Jonathan Lee. Dueling rl: reinforcement learning with trajectory preferences. *arXiv preprint arXiv:2111.04850*, 2021.

Richard Yuanzhe Pang, Weizhe Yuan, He He, Kyunghyun Cho, Sainbayar Sukhbaatar, and Jason Weston. Iterative reasoning preference optimization. *Advances in Neural Information Processing Systems*, 37:116617–116637, 2024.

Jiahao Qiu, Yifu Lu, Yifan Zeng, Jiacheng Guo, Jiayi Geng, Huazheng Wang, Kaixuan Huang, Yue Wu, and Mengdi Wang. Treebon: Enhancing inference-time alignment with speculative tree-search and best-of-n sampling. *arXiv preprint arXiv:2410.16033*, 2024.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36:53728–53741, 2023.

Rafael Rafailov, Joey Hejna, Ryan Park, and Chelsea Finn. From r to q^*: Your language model is secretly a Q-function. *arXiv preprint arXiv:2404.12358*, 2024.

Rajkumar Ramamurthy, Prithviraj Ammanabrolu, Kianté Brantley, Jack Hessel, Rafet Sifa, Christian Bauckhage, Hannaneh Hajishirzi, and Yejin Choi. Is reinforcement learning (not) for natural language processing: Benchmarks, baselines, and building blocks for natural language policy optimization, 2023. URL `https://arxiv.org/abs/2210.01241`.

Dorsa Sadigh, Anca Dragan, Shankar Sastry, and Sanjit Seshia. *Active preference-based learning of reward functions*. CoRR, 2017.

Nihar B Shah, Sivaraman Balakrishnan, Joseph Bradley, Abhay Parekh, Kannan Ramch, Martin J Wainwright, et al. Estimation from pairwise comparisons: Sharp minimax bounds with topology dependence. *Journal of Machine Learning Research*, 17(58):1–47, 2016.

Avi Singh, John D Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J Liu, James Harrison, Jaehoon Lee, Kelvin Xu, et al. Beyond human data: Scaling self-training for problem-solving with language models. *arXiv preprint arXiv:2312.06585*, 2023.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. Learning to summarize with human feedback. *Advances in Neural Information Processing Systems*, 33:3008–3021, 2020.

Shubham Toshniwal, Wei Du, Ivan Moshkov, Branislav Kisacanin, Alexan Ayrapetyan, and Igor Gitman. Openmathinstruct-2: Accelerating ai for math with massive open-source instruction data. *arXiv preprint arXiv:2410.01560*, 2024.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Peiyi Wang, Lei Li, Zhihong Shao, RX Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. *arXiv preprint arXiv:2312.08935*, 2023.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.

Chenlu Ye, Wei Xiong, Yuheng Zhang, Hanze Dong, Nan Jiang, and Tong Zhang. Online iterative reinforcement learning from human feedback with general preference model. *Advances in Neural Information Processing Systems*, 37:81773–81807, 2024.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D Goodman. Star: Self-taught reasoner bootstrapping reasoning with reasoning. In *Proc. the 36th International Conference on Neural Information Processing Systems*, volume 1126, 2024.

Dan Zhang, Sining Zhoubian, Ziniu Hu, Yisong Yue, Yuxiao Dong, and Jie Tang. Rest-mcts*: Llm self-training via process reward guided tree search. *Advances in Neural Information Processing Systems*, 37:64735–64772, 2024.

Shun Zhang, Zhenfang Chen, Yikang Shen, Mingyu Ding, Joshua B Tenenbaum, and Chuang Gan. Planning with large language models for code generation. *arXiv preprint arXiv:2303.05510*, 2023.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning acting and planning in language models. *arXiv preprint arXiv:2310.04406*, 2023.

Jin Peng Zhou, Kaiwen Wang, Jonathan Chang, Zhaolin Gao, Nathan Kallus, Kilian Q Weinberger, Kianté Brantley, and Wen Sun. Q#: Provably optimal distributional RL for LLM post-training. *arXiv preprint arXiv:2502.20548*, 2025.

Banghua Zhu, Michael Jordan, and Jiantao Jiao. Principled reinforcement learning with human feedback from pairwise or k-wise comparisons. In *International Conference on Machine Learning*, pages 43037–43067. PMLR, 2023.

Daniel M Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.